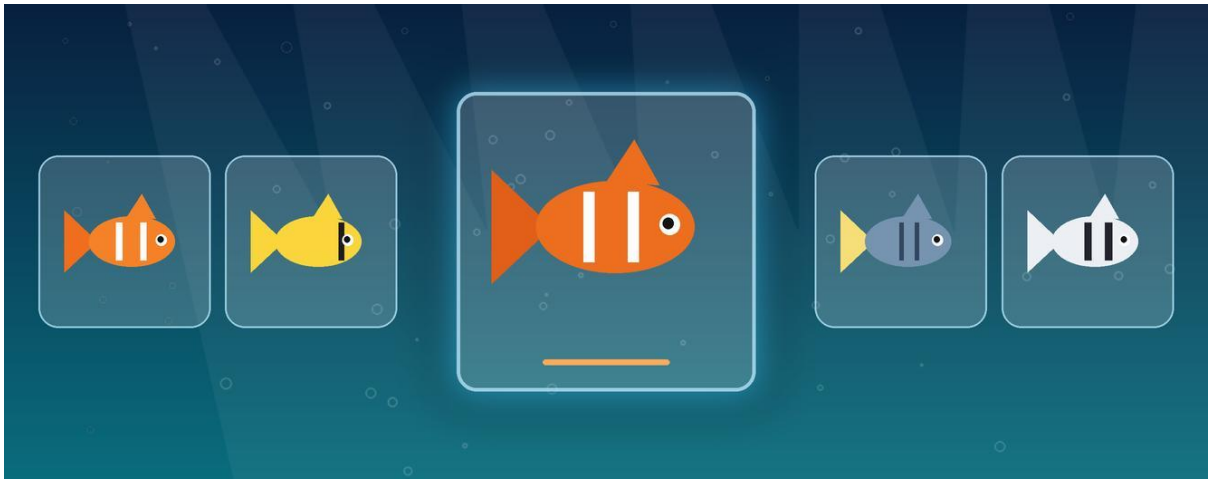
SUPERIOR UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE

AquaClass AI

Fish Species Classification System



SUBMITTED BY

Najamul Hassan

Roll No.: SU92-BSCSM-F23-665

COURSE: Deep Learning

SECTION: 7B

SUBMITTED TO

Dr. Munir Ahmad

Abstract

AquaClass AI is a deep learning system that identifies the species of coral reef fish from an uploaded image. A **MobileNetV2** convolutional neural network (CNN), pretrained on ImageNet, was adapted by transfer learning to classify **five species** from the Fish4Knowledge dataset, which contains **24,971 images**. On the held-out test set of 3,750 images the model reaches **94.69% accuracy** (weighted precision 94.66%, weighted recall 94.69%, weighted F1-score 94.66%). Performance is highest for *Chaetodon lunulatus* (F1-score 98.69%) and lowest for *Chromis chrysura* (F1-score 86.23%), which is most often confused with *Dascyllus reticulatus*. The trained model was exported to the ONNX format and is served by a **FastAPI** backend that runs it with **ONNX Runtime**. A **React** web application sends the uploaded image to the backend and displays the returned prediction. The application shows the predicted species, a confidence score, the top three predictions, dataset statistics, model architecture details and evaluation results.

1. Introduction

1.1 Problem statement

Automated fish species classification plays a useful role in marine biology, aquaculture monitoring and biodiversity preservation. Identification by marine taxonomists is accurate but labour-intensive, and it is difficult to scale to the continuous underwater video that modern monitoring systems record.

The task is hard for a computer because underwater images vary in lighting, water clarity, fish pose and background, and because several species share similar body shapes and colour patterns. The goal of this project is to build a classifier that, given an image of a fish, outputs the species name and how confident the model is, and to make this available through a simple web interface backed by an API that runs the model.

1.2 Objectives

1. Train a CNN with transfer learning that classifies five coral reef fish species with high accuracy.
2. Evaluate the model with accuracy, precision, recall, F1-score and a confusion matrix on images that were not used for training.
3. Export the trained model to the ONNX format and serve it through a FastAPI backend that uses ONNX Runtime, so that the React web application can request predictions through an API.
4. Build a client-server web application (React frontend and FastAPI backend) where a user can upload an image, see the prediction returned by the backend, and explore the dataset, the model architecture and the evaluation results.

1.3 Tools and technologies

Area	Tools
Language	Python (model training and backend), JavaScript (frontend)
Deep learning	PyTorch, MobileNetV2 pretrained on ImageNet
Inference engine	ONNX Runtime (server-side), ONNX model format (.onnx)
Backend	FastAPI (Python), Pillow (PIL) and NumPy for image preprocessing
Frontend	React, Vite, JavaScript, Tailwind CSS, Recharts, Lucide icons
Hosting	Vercel (frontend and backend deployed separately)
Training environment	Google Colab GPU

2. Dataset

2.1 Fish4Knowledge

The project uses the **Fish4Knowledge** ground-truth recognition dataset (Boom et al., 2014), collected by the University of Edinburgh from underwater camera footage of coral reefs. It consists of cropped, expert-verified images of individual fish. Five species (classes) were used, giving 24,971 images in total.

Property	Value	Note
Number of classes	5	Coral reef fish species
Total images	24,971	All five classes
Training images	17,478	70%
Validation images	3,743	15%
Test images	3,750	15%, unseen during training
Model input size	224 x 224 x 3	RGB, after resizing

Dataset link:
https://homepages.inf.ed.ac.uk/rbf/Fish4Knowledge/GROUNDTRUTH/RECOG/

2.2 Classes and data split

Table 1 lists the five species with their image counts. The split follows a 70/15/15 ratio for each class, so every species keeps the same share in the training, validation and test sets.

Table 1: Images per species and data split.

Species	Total	Train	Validation	Test
Amphiprion clarkii	4,049	2,834	607	608
Chaetodon lunulatus	2,534	1,773	380	381
Chromis chrysura	3,593	2,515	538	540
Dascyllus reticulatus	12,112	8,478	1,816	1,818
Plectroglyphidodon dickii	2,683	1,878	402	403
Total	**24,971**	**17,478**	**3,743**	**3,750**



Figure 1: Class distribution of the dataset.

2.3 Species description

Species	Key visual features	Habitat	Depth	Size
Amphiprion clarkii – Clark's Anemonefish	Dark brown or black body with two or three white bands; territorial; lives with sea anemones	Tropical reefs and shallow lagoons	1–60 m	10–15 cm
Chaetodon lunulatus – Oval Butterflyfish	Yellow oval body with thin dark lines and a black eye stripe; feeds on coral polyps	Coral-rich reefs and protected bays	3–30 m	13–15 cm
Chromis chrysura – Stout-body Chromis	Metallic blue-grey body, dark vertical stripes, pale yellow tail	Outer reef slopes and rocky substrates	6–45 m	12–16 cm
Dascyllus reticulatus – Reticulate Dascyllus	White body, two dark vertical bars, black-edged scales forming a mesh pattern	Branching Acropora coral colonies	1–50 m	8–9 cm
Plectroglyphidodon dickii – Dick's Damselfish	Brownish-yellow front body, bold black band near the rear, white tail base	Exposed surge zones and reef tops	1–15 m	9–11 cm

2.4 Characteristics and challenges

- **Class imbalance:** *Dascyllus reticulatus* makes up 48.5% of all images (12,112), while *Chaetodon lunulatus* makes up only 10.1% (2,534). The largest class is about 4.8 times larger than the smallest.
- **Visually similar species:** four of the five species are damselfish or anemonefish with similar body shapes and stripe patterns, which makes some pairs hard to separate.
- **Underwater image quality:** images come from video footage, so lighting, water clarity, pose and image size vary from fish to fish.

3. Methodology

3.1 Preprocessing

1. Each image is resized to 224 x 224 pixels, the input size expected by MobileNetV2. During training and evaluation this is done in the PyTorch data pipeline. At inference time the FastAPI backend applies the same resizing using Python, PIL (Pillow) and NumPy.
2. The pixel values are scaled to the range 0–1, arranged channels first, and normalized with the ImageNet statistics: mean = [0.485, 0.456, 0.406] and standard deviation = [0.229, 0.224, 0.225]. The pretrained backbone was trained with this normalization, so the same values are used in training, evaluation and inference. In the deployed application this step runs on the server: the backend converts the uploaded image to RGB, resizes it to 224 x 224 with PIL, applies the ImageNet normalization with NumPy, and passes the resulting float tensor to ONNX Runtime. The frontend does not preprocess or normalize the image.
3. The data is divided into 70% training, 15% validation and 15% test images.

3.2 Data augmentation

To reduce overfitting, the training images are randomly transformed in every epoch:

- RandomHorizontalFlip
- RandomRotation of up to 15 degrees

Horizontal flipping is safe for this task because a fish is the same species whether it faces left or right. Validation and test images are not augmented.

3.3 Transfer learning and model architecture

Training a deep CNN from scratch needs large amounts of data and computation. Transfer learning (Pan and Yang, 2010) reuses a network that has already learned general visual features, such as edges, textures and shapes, from a large dataset. In this project a **MobileNetV2** (Sandler et al., 2018) pretrained on ImageNet is used as a frozen feature extractor, and only a new classification head is trained on the fish images.

Table 2: Model architecture.

Block	Layers
Input	224 x 224 x 3 RGB image, ImageNet normalization
Backbone	MobileNetV2 convolutional feature extractor (ImageNet pretrained, frozen); outputs 1,280 features
Classifier head	Dropout(0.2), Linear(1280 → 5)
Output	Softmax over the 5 species; top-3 probabilities are shown as confidence scores

MobileNetV2 was chosen because it is accurate while staying small and fast, which suits deployment in a web application. Its efficiency comes from three ideas:

- **Depthwise separable convolutions** greatly reduce the number of parameters and computations while keeping good feature extraction quality.
- **Inverted residual blocks** connect narrow bottleneck layers directly, which saves memory and helps gradients flow.
- **Linear bottlenecks** avoid applying non-linearities in low-dimensional layers, which would destroy useful information.



Figure 2: Classification pipeline.

3.4 Training configuration

Setting	Value
Base architecture	MobileNetV2 (ImageNet pretrained), backbone frozen
Trained part	Classification head: Dropout(0.2) + Linear(1280 → 5)
Input resolution	224 x 224 x 3 (RGB)
Optimizer	Adam (learning rate = 0.001)
Loss function	CrossEntropyLoss
Batch size	32 images per batch
Epochs	10
Data augmentation	RandomHorizontalFlip, RandomRotation(15)
Best validation accuracy	95.03% (epoch 7)
Checkpoint	Best weights saved as aquaclass_mobilenetv2_best.pth
Export format	ONNX (.onnx), exported from the PyTorch checkpoint
Inference runtime	ONNX Runtime (server-side, FastAPI backend)

After each epoch the model is evaluated on the validation set, and the weights with the best validation accuracy (95.03%, reached at epoch 7) are kept as the checkpoint. The test set is used only once, for the final evaluation.

After training, the best checkpoint was exported to the ONNX format (.onnx) so that it can be loaded by ONNX Runtime in the backend. The exported model keeps the same architecture and weights; only the file format changes. Training was done in PyTorch, and PyTorch is not needed when the application runs.

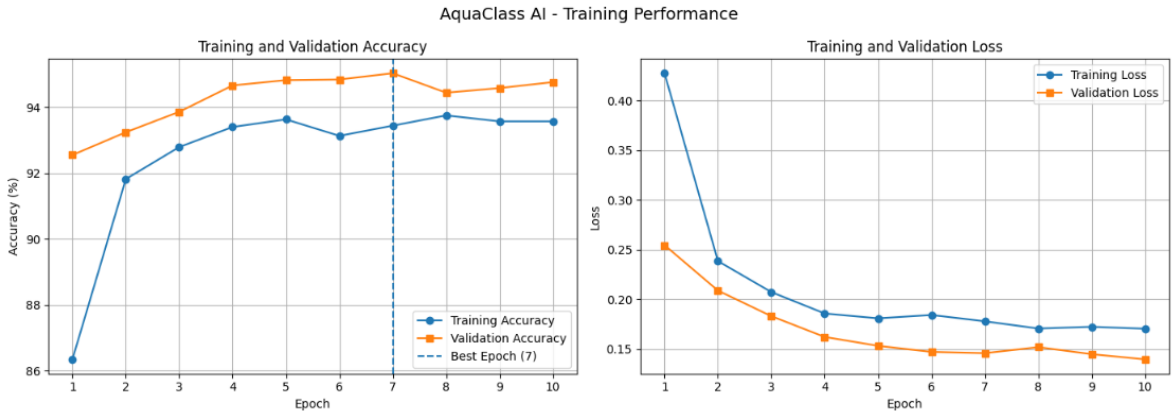


Figure 3: Training and validation accuracy and loss curves.

4. Web Application

4.1 System architecture

The system uses a **client-server architecture** with two parts that are deployed separately on Vercel. The **React** frontend (built with Vite and JavaScript) provides the interface. When the user uploads an image, the frontend sends it to the **FastAPI** backend through an API request. The backend preprocesses the image, runs the exported MobileNetV2 model with **ONNX Runtime**, and returns the predicted species, the confidence score and the top-three predictions as a response, which the frontend then displays. The address of the backend is set in the frontend with the environment variable VITE_API_URL.

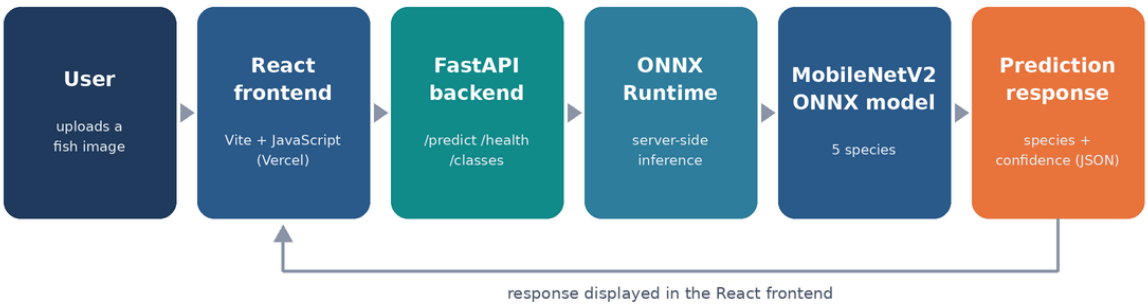


Figure 4: System architecture.

4.2 Model export and backend inference

Step	Component	Purpose
Export	PyTorch → ONNX	The trained checkpoint (aquaclass_mobilenetv2_best.pth) is exported to the ONNX format (.onnx) so that it can run outside PyTorch.
Load	ONNX Runtime	The FastAPI backend loads the .onnx file and creates an inference session.
Preprocess	Python, PIL, NumPy	The uploaded image is resized to 224 x 224, normalized with the ImageNet mean and standard deviation, and stored as a float tensor.
Infer	ONNX Runtime session	The tensor is passed to the model, which returns one score for each of the five species.
Output	Softmax	The scores are converted to probabilities; the predicted species, its confidence and the top-3 predictions are returned to the frontend.

The application consists of two code bases: the React frontend (pages, components and the code that calls the API) and the FastAPI backend (the API endpoints, the preprocessing code, the ONNX Runtime session and the .onnx model file).

4.3 Frontend pages

The interface supports light and dark mode and is organised into six pages:

Page	Content
Home	Project statistics (dataset size, number of species, test accuracy, architecture), a four-step explanation of how classification works, and cards for the five species.
AI Classifier	Upload an image by drag and drop or choose one of five preset samples. An uploaded image is sent to the FastAPI backend through the /predict endpoint, and the page shows the predicted species, confidence score and top-3 predictions returned by the backend, together with species information stored in the frontend. A label on the result shows whether it came from the FastAPI service (see Section 5.5).
Dataset Explorer	Class imbalance chart, training/validation/test split cards and a gallery of the five species.
Model Architecture	MobileNetV2 design ideas, training hyperparameters and the three-step pipeline.
Evaluation	Accuracy, precision, recall and F1 cards, the class-level report table and a 5 x 5 confusion matrix heatmap.
Resources & About	Links to the source code, project report, presentation and dataset, and an API Config dialog for setting the backend address; a project overview.

4.4 Model loading and prediction

The backend loads the ONNX model once, when it starts, and keeps the ONNX Runtime inference session in memory so that it can be reused for every request. It provides three endpoints, which the frontend and any other client can call over HTTP:

Endpoint	Purpose
/health	GET request. Returns a simple status response (status "ok" and a model_loaded flag) to confirm that the backend is running with the model loaded. The frontend calls it when the page opens, and whenever the backend address changes, to show the connection status badge in the header.
/classes	GET request. Returns the list of the five fish species (class names) that the model can predict.
/predict	POST request. Receives an uploaded image file, checks that it is an image, preprocesses it (Python, PIL, NumPy), runs the ONNX model and returns the prediction as JSON (see below).

The /predict endpoint returns three items: the predicted species (predicted_class), its confidence score (confidence, the softmax probability of the top class) and the three most probable species with their confidence scores (top_predictions). If the uploaded file is not an image, or the prediction fails, the endpoint returns an error response with HTTP status 400 and a short message. Because the prediction is computed on the server, the image is sent to the backend over the network and the prediction time depends on the backend and the network connection.

4.5 Running the application

1. **Backend:** open the backend folder, install the Python packages listed in requirements.txt, and start the FastAPI server (for example with uvicorn). The backend loads the ONNX model and exposes /health, /classes and /predict.
2. **Frontend:** open the frontend folder, install the packages with npm install, and set the environment variable VITE_API_URL to the address of the backend (for example the local backend address, or the deployed backend URL). If VITE_API_URL is not set, the frontend uses http://localhost:8000 as its default backend address. The Resources & About page also has an API Config dialog in which the backend address can be changed while the application is running, without rebuilding the frontend.
3. **Run:** start the frontend with npm run dev, open http://localhost:5173 in the browser, upload a fish image and select Identify Fish Species.

Deployment: the frontend and the backend are two separate projects, each deployed separately on Vercel (frontend: https://aquaclass-aii.vercel.app/, backend: https://aquaclass-ai-backend.vercel.app). In the deployed frontend, VITE_API_URL points to the deployed backend instead of a local address. The complete set of project links is given in Section 10.

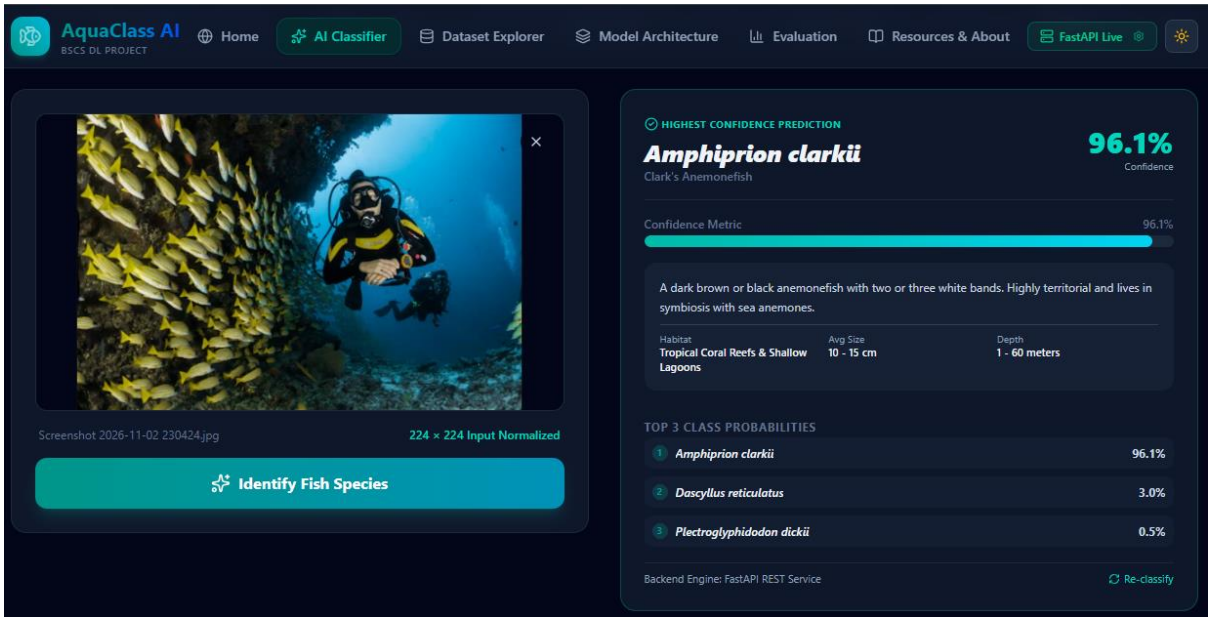
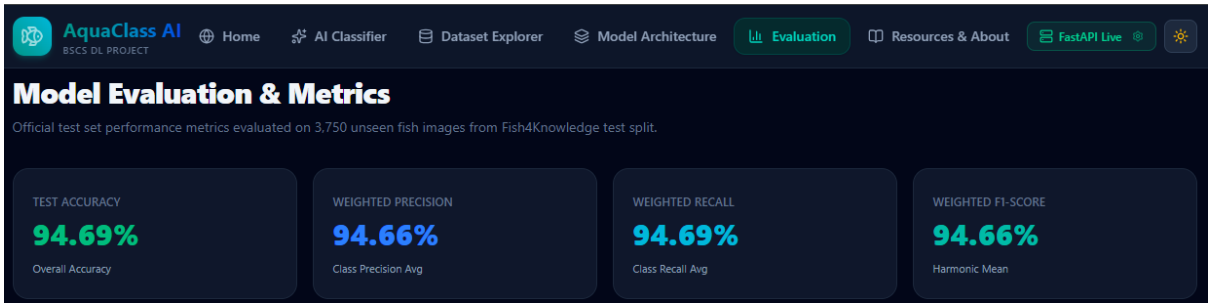
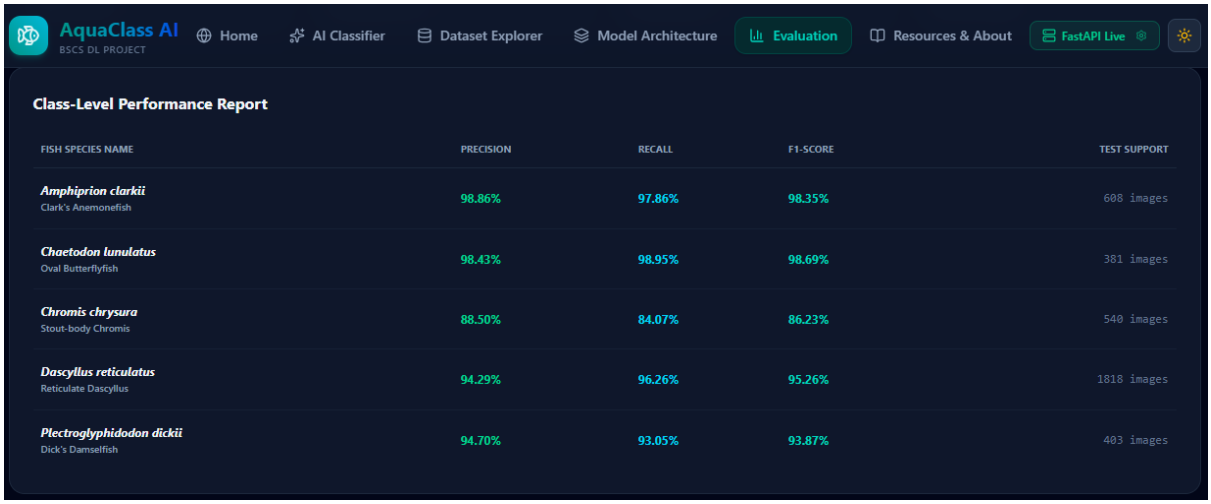
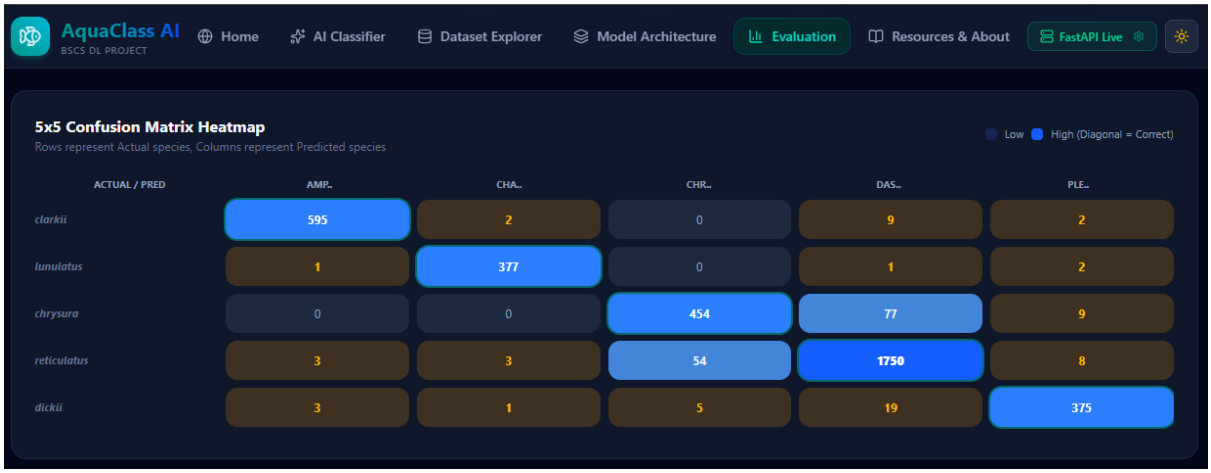


Figure 5: AI Classifier page showing a prediction returned by the FastAPI backend for an uploaded image.


AquaClass AI
BSCS DL PROJECT
Home
AI Classifier
Dataset Explorer
Model Architecture
Evaluation
Resources & About
FastAPI Live
Model Evaluation & Metrics
Official test set performance metrics evaluated on 3,750 unseen fish images from Fish4Knowledge test split.
TEST ACCURACY
94.69%
Overall Accuracy
WEIGHTED PRECISION
94.66%
Class Precision Avg
WEIGHTED RECALL
94.69%
Class Recall Avg
WEIGHTED F1-SCORE
94.66%
Harmonic Mean


AquaClass AI
BSCS DL PROJECT
Home
AI Classifier
Dataset Explorer
Model Architecture
Evaluation
Resources & About
FastAPI Live
Class-Level Performance Report
FISH SPECIES NAME
PRECISION
RECALL
F1-SCORE
TEST SUPPORT
Amphiprion clarkii
Clark's Anemonefish
98.86%
97.86%
98.35%
608 images
Chaetodon lunulatus
Oval Butterflyfish
98.43%
98.95%
98.69%
381 images
Chromis chrysura
Stout-body Chromis
88.50%
84.07%
86.23%
540 images
Dascyllus reticulatus
Reticulate Dascyllus
94.29%
96.26%
95.26%
1818 images
Plectroglyphidodon dickii
Dick's Damselfish
94.70%
93.05%
93.87%
403 images


AquaClass AI
BSCS DL PROJECT
Home
AI Classifier
Dataset Explorer
Model Architecture
Evaluation
Resources & About
FastAPI Live
5x5 Confusion Matrix Heatmap
Rows represent Actual species, Columns represent Predicted species
Low
High (Diagonal = Correct)
ACTUAL / PRED
AMP..
CHA..
CHR..
DAS..
PLE..
clarkii
595
2
0
9
2
lunulatus
1
377
0
1
2
chrysura
0
0
454
77
9
reticulatus
3
3
54
1750
8
dickii
3
1
5
19
375

5. Results

5.1 Overall performance

All metrics below were measured on the **test set of 3,750 images**, which was not used for training or for choosing the best epoch.

Table 3: Overall test performance.

Metric	Value
Test accuracy	94.69%
Weighted precision	94.66%
Weighted recall	94.69%
Weighted F1-score	94.66%
Best validation accuracy	95.03% (epoch 7)

5.2 Class-level performance

Table 4: Precision, recall and F1-score per species (test set).

Species	Precision	Recall	F1-score
Amphiprion clarkii	98.84%	97.86%	98.35%
Chaetodon lunulatus	98.43%	98.95%	98.69%
Chromis chrysura	88.50%	84.07%	86.23%
Dascyllus reticulatus	94.29%	96.26%	95.26%
Plectroglyphidodon dickii	94.70%	93.05%	93.87%
Macro average (computed from the rows above)	94.95%	94.04%	94.48%

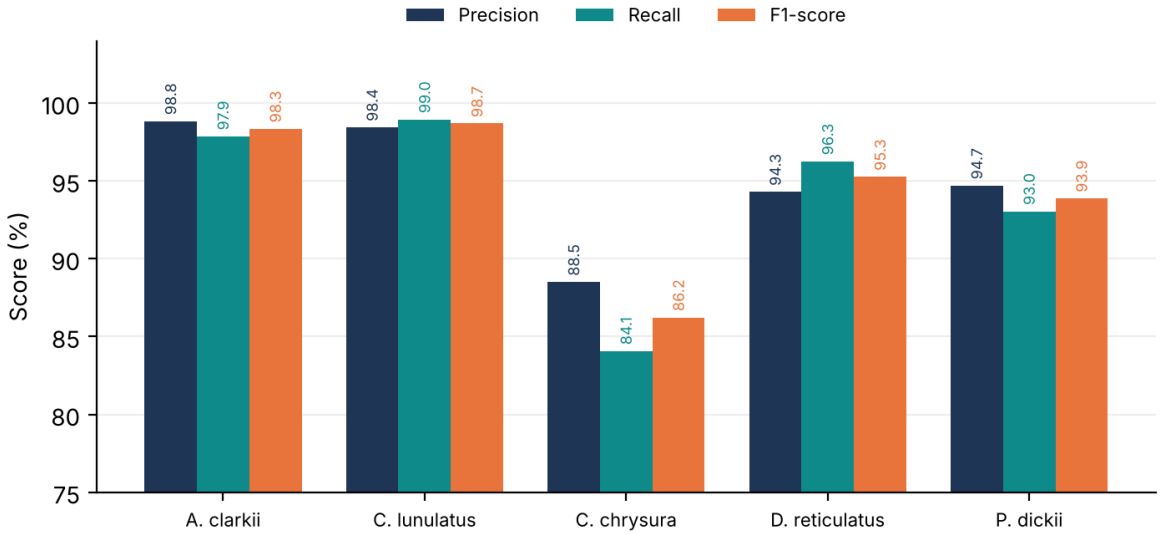


Figure 7: Per-class precision, recall and F1-score.

Four of the five species reach an F1-score above 93%. *Chaetodon lunulatus* is recognized almost perfectly (98.69%), probably because its yellow oval body with dark lines and eye stripe is distinctive. *Chromis chrysura* is clearly the weakest class: its recall of 84.07% means that about 16% of its test images (86 of 540) were assigned to another species.

5.3 Confusion matrix and error analysis

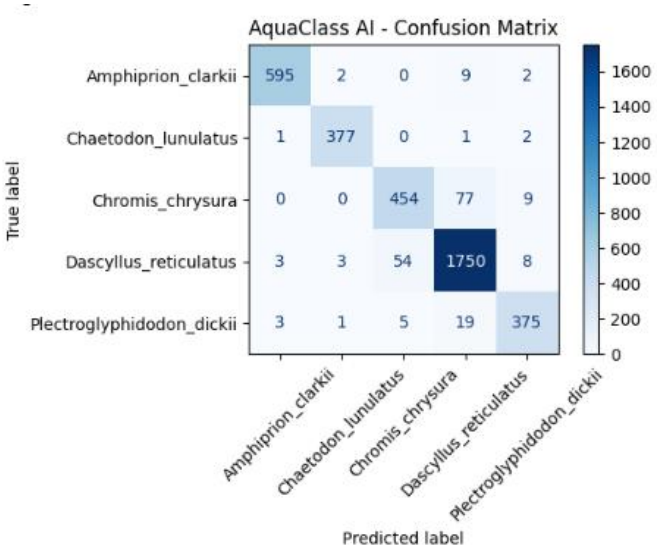


Figure 8: Confusion matrix on the test set (rows: actual species, columns: predicted species).

The model classified 3,551 of the 3,750 test images correctly (94.69%) and made 199 errors. The errors are concentrated between two classes:

- 77 images of *Chromis chrysura* (14.3% of its 540 test images) were predicted as *Dascyllus reticulatus*, and 54 images of *Dascyllus reticulatus* (3.0% of 1,818) were predicted as *Chromis chrysura*. This single pair accounts for 131 of the 199 errors.
- Smaller confusions exist from *Chromis chrysura* to *Plectroglyphidodon dickii* (9 images) and from *Plectroglyphidodon dickii* to *Dascyllus reticulatus* (19 images).

Two likely reasons are the visual similarity of these damselfish and the class imbalance: *Dascyllus reticulatus* has 3.4 times more images than *Chromis chrysura*, so the model sees many more examples of it and tends to predict it when unsure. These explanations were not tested separately in this project.

5.4 Validation versus test accuracy

The best validation accuracy (95.03%, epoch 7) and the final test accuracy (94.69%) are close, differing by only 0.34 percentage points. The small gap indicates that the model generalizes to unseen images and that the epoch chosen on the validation set did not overfit to it.

5.5 Testing of the web application

The following functional tests were performed manually on the AquaClass AI application (frontend and backend). They are grouped by what each test checks. These tests verify that the application works; the measured performance of the model itself is the test-set evaluation in Sections 5.1 to 5.3.

1. **Manual image upload testing:** Fish images of all five species were uploaded in the AI Classifier page, and the predicted species and confidence score were checked against the known species of each image. These uploads are the tests that exercise the real backend path (frontend → /predict → ONNX Runtime).
2. **Preset sample testing:** The five preset sample images were selected in the AI Classifier page to check image selection, preview and display of the result. In the current frontend

code a preset sample is chosen by its image address rather than uploaded as a file, so it is not sent to the /predict endpoint. Preset samples therefore test the interface only and are not evidence of model accuracy.

3. **Backend model loading testing:** The FastAPI backend was started and checked to confirm that the ONNX model file is found and loaded into an ONNX Runtime session without errors.
4. **API endpoint testing:** The /health, /classes and /predict endpoints were called directly and checked to confirm that each returns the expected response described in Section 4.4.
5. **Frontend–backend integration testing:** The React frontend, configured with VITE_API_URL, was checked to confirm that it contacts the backend, sends the uploaded image to /predict, and displays the returned species, confidence score and top-three predictions. The result panel states whether the result came from the FastAPI service or from the frontend's built-in demonstration fallback (see Section 7).
6. **Interface testing:** The upload control, result display, page navigation, light and dark modes and overall responsiveness of the interface were checked.

Testing result: These were qualitative functional checks. Their observations were used to find prediction problems, interface problems, backend and API errors and model-loading errors. No response-time measurements are reported, and no quantitative accuracy is claimed from the manual tests; the quantitative results of the project are those in Sections 5.1 to 5.3.

6. Challenges and Iterations

1. **Class imbalance.** One class contains almost half of the dataset. The class-level report is therefore used together with overall accuracy, because accuracy alone can hide weak performance on the smaller classes.
2. **Similar species.** *Chromis chrysura* and *Dascyllus reticulatus* are frequently confused (Section 5.3). This is the main source of error and the main target for future improvement.
3. **Backend integration.** A PyTorch model cannot be used directly in a web page, so the trained checkpoint was exported to the ONNX format and served by a FastAPI backend with ONNX Runtime. The backend preprocessing (Python, PIL and NumPy) had to match the ImageNet normalization used in training so that the predictions are consistent.
4. **API configuration.** The frontend and the backend are separate applications, so the frontend must know the backend address. This is set with the environment variable VITE_API_URL, and the backend must accept requests from the frontend origin (CORS).
5. **Deployment.** The frontend and the backend were deployed as two separate projects on Vercel. The backend deployment had to include the ONNX model file and its dependencies, and the deployed frontend had to be pointed to the deployed backend instead of a local address.

7. Limitations

- The model knows **five species only**. It always predicts one of them, even for a fish or an object that belongs to none of the classes, because there is no “unknown” class.
- Only the classification head was trained. The backbone stayed frozen, so features specific to fish images were not learned.
- All images come from a single dataset. Performance on other cameras, locations, water conditions or video frames was not measured.
- The classifier expects one fish per image. It does not detect and locate fish in a full underwater scene.
- The confidence value is the softmax probability. It is useful but is not a calibrated measure of certainty.
- Class imbalance remains and was not corrected during training.
- Predictions need a network connection to the backend, so the application cannot classify images when the backend is unavailable, and the response time depends on the backend and the network.
- The frontend contains a built-in demonstration fallback that is used when the backend cannot be reached or returns an error, and for the preset samples. Its output is not a model prediction. The result panel labels the source (FastAPI service or client fallback), and only results labelled as coming from the FastAPI service should be treated as model predictions.

8. Future Work

- Add a **detection** stage (for example YOLO) so fish can be found and classified in video frames.
- Include more species and an “unknown” class so that unrelated inputs can be rejected.
- Fine-tune the last blocks of the backbone, or compare other lightweight architectures such as EfficientNet.
- Reduce the effect of imbalance with a class-weighted loss or oversampling of the smaller classes.
- Add visual explanations such as Grad-CAM to show which image regions influence a prediction.
- Reduce the size of the ONNX model (for example with quantization) so that the backend loads and responds faster, and build a mobile version.

9. Conclusion

AquaClass AI shows that transfer learning with a lightweight MobileNetV2 network can classify five coral reef fish species with 94.69% accuracy on an unseen test set of 3,750 images, with weighted precision, recall and F1-score all close to 95%. The error analysis shows that the remaining mistakes are concentrated between *Chromis chrysura* and *Dascyllus reticulatus*, which points to class imbalance and visual similarity as the main limits. The project covers the complete workflow: data preparation, model design, training, evaluation, export to ONNX, and

an interactive client-server web application, with a React frontend and a FastAPI backend that runs the model with ONNX Runtime, for exploring the data, the model and the results.

10. Project Links

Resource	Link
Web application (Frontend)	https://aquaclass-aii.vercel.app/ (React frontend on Vercel)
Backend API	https://aquaclass-ai-backend.vercel.app (FastAPI backend on Vercel)
GitHub repository (code)	https://github.com/chnajamulhassan/aquaclass-aii
Dataset (Fish4Knowledge)	https://homepages.inf.ed.ac.uk/rbf/Fish4Knowledge/GROUNDTRUTH/RECOG/
Presentation	https://docs.google.com/presentation/d/1kPWjWiFhslrYwkWuA_rxealszJzKtdzy/edit?usp=sharing&ouid=
Notebook	https://colab.research.google.com/drive/1Mhy_d5j4OG_csM40VDczVXnG_n5u3x-m?usp=sharing&utm_

References

1. Boom, B. J., He, J., Palazzo, S., Huang, P. X., Beyan, C., Chou, H.-M., Lin, F.-P., Spampinato, C., and Fisher, R. B. (2014). A research tool for long-term and continuous analysis of fish assemblage in coral-reefs using underwater camera footage. *Ecological Informatics*, 23, 83–97.
2. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. *CVPR*, 4510–4520.
3. Howard, A. G. et al. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv:1704.04861.
4. Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. (2009). ImageNet: A Large-Scale Hierarchical Image Database. *CVPR*.
5. Pan, S. J. and Yang, Q. (2010). A Survey on Transfer Learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10), 1345–1359.
6. Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*.
7. Kingma, D. P. and Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR*.
8. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.
9. ONNX Runtime documentation. https://onnxruntime.ai/docs/
10. ONNX: Open Neural Network Exchange. https://onnx.ai